Temporal tables

Applies to: SQL Server 2016 (13.x) and later versions Azure SQL Database Azure SQL Managed Instance SQL database in Microsoft Fabric

Temporal tables (also known as system-versioned temporal tables), are a database feature that brings built-in support for providing information about data stored in the table at any point in time, rather than only the data that is correct at the current moment in time.

Get started with system-versioned temporal tables, and review Temporal table usage scenarios.

What is a system-versioned temporal table?

A system-versioned temporal table is a type of user table designed to keep a full history of data changes, allowing easy point-in-time analysis. This type of temporal table is referred to as a system-versioned temporal table, because the system manages the period of validity for each row (that is, the Database Engine).

Every temporal table has two explicitly defined columns, each with a **datetime2** data type. These columns are referred to as *period* columns. These period columns are used exclusively by the system to record the period of validity for each row, whenever a row is modified. The main table that stores current data is referred to as the *current table*, or simply as the *temporal table*.

In addition to these period columns, a temporal table also contains a reference to another table with a mirrored schema, called the *history table*. The system uses the history table to automatically store the previous version of the row each time a row in the temporal table gets updated or deleted. During temporal table creation, you can specify an existing history table (which must be schema compliant) or let the system create a default history table.

Why temporal?

Real data sources are dynamic and more often than not business decisions rely on insights that analysts can get from data evolution. Use cases for temporal tables include:

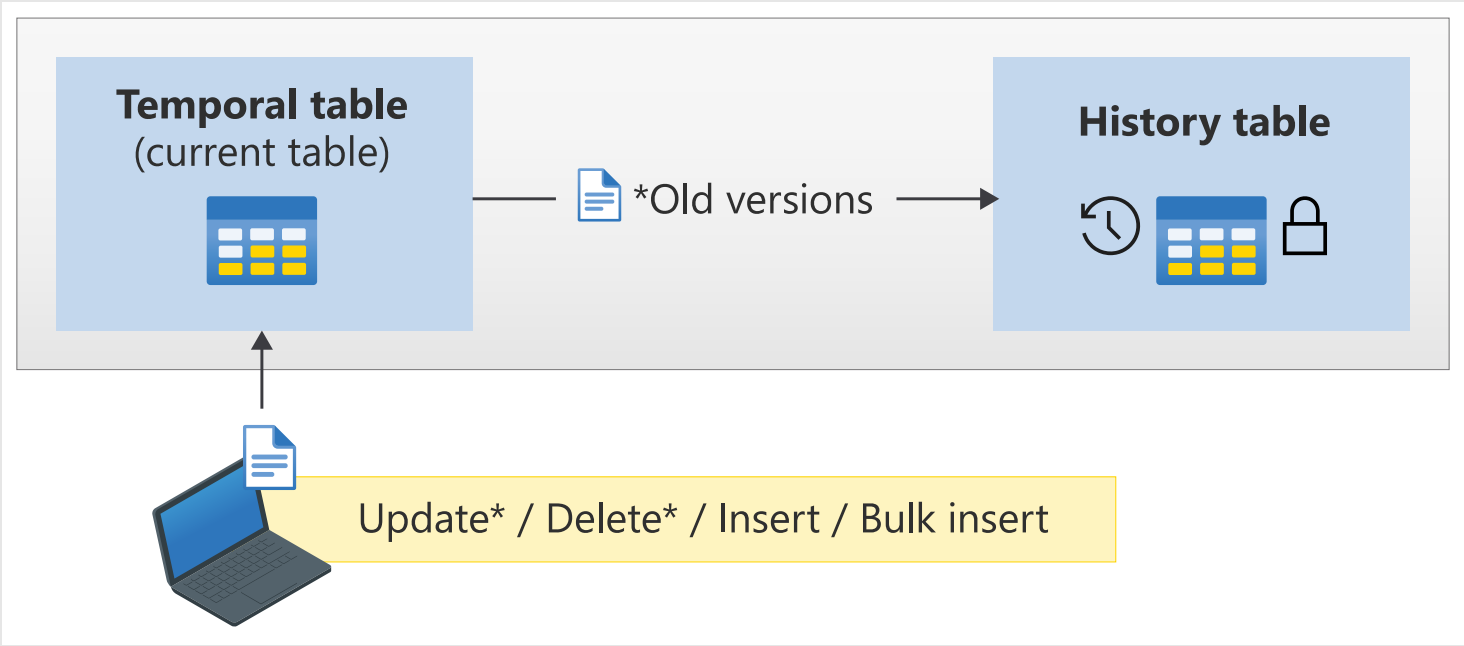
- Auditing all data changes and performing data forensics when necessary
- Reconstructing state of the data as of any time in the past
- Calculating trends over time
- Maintaining a slowly changing dimension for decision support applications
- Recovering from accidental data changes and application errors

How does temporal work?

System-versioning for a table is implemented as a pair of tables: a current table, and a history table. Within each of these tables, two extra **datetime2** columns are used to define the period of validity for each row:

- **Period start column**: The system records the start time for the row in this column, typically denoted as the `ValidFrom` column.
- **Period end column**: The system records the end time for the row in this column, typically denoted as the `ValidTo` column.

The current table contains the *current value* for each row. The history table contains each previous value (the *old version*) for each row, if any, and the start time and end time for the period for which it was valid.



The following script illustrates a scenario with employee information:

SQL

```
CREATE TABLE dbo.Employee
(
    [EmployeeID] INT NOT NULL PRIMARY KEY CLUSTERED,
    [Name] NVARCHAR (100) NOT NULL,
    [Position] VARCHAR (100) NOT NULL,
    [Department] VARCHAR (100) NOT NULL,
    [Address] NVARCHAR (1024) NOT NULL,
    [AnnualSalary] DECIMAL (10, 2) NOT NULL,
    [ValidFrom] DATETIME2 GENERATED ALWAYS AS ROW START,
    [ValidTo] DATETIME2 GENERATED ALWAYS AS ROW END,
    PERIOD FOR SYSTEM_TIME (ValidFrom, ValidTo)
)
WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = dbo.EmployeeHistory));
```

For more information, see Create a system-versioned temporal table.

- **Inserts:** The system sets the value for the `ValidFrom` column to the begin time of the current transaction (in the UTC time zone) based on the system clock and assigns the value for the `ValidTo` column to the maximum value of `9999–12–31`. This marks the row as open.

- **Updates:** The system stores the previous value of the row in the history table and sets the value for the `ValidTo` column to the begin time of the current transaction (in the UTC time zone) based on the system clock. This marks the row as closed, with a period recorded for which the row was valid. In the current table, the row is updated with its new value and the system sets the value for the `ValidFrom` column to the begin time for the transaction (in the UTC time zone) based on the system clock. The value for the updated row in the current table for the `ValidTo` column remains the maximum value of `9999-12-31`.

- **Deletes:** The system stores the previous value of the row in the history table and sets the value for the `ValidTo` column to the begin time of the current transaction (in the UTC time zone) based on the system clock. This marks the row as closed, with a period recorded for which the previous row was valid. In the current table, the row is removed. Queries of the current table don't return this row. Only queries that deal with history data return data for which a row is closed.

- **Merge:** The operation behaves exactly as if up to three statements (an `INSERT`, an `UPDATE`, and/or a `DELETE`) executed, depending on what is specified as actions in the `MERGE` statement.

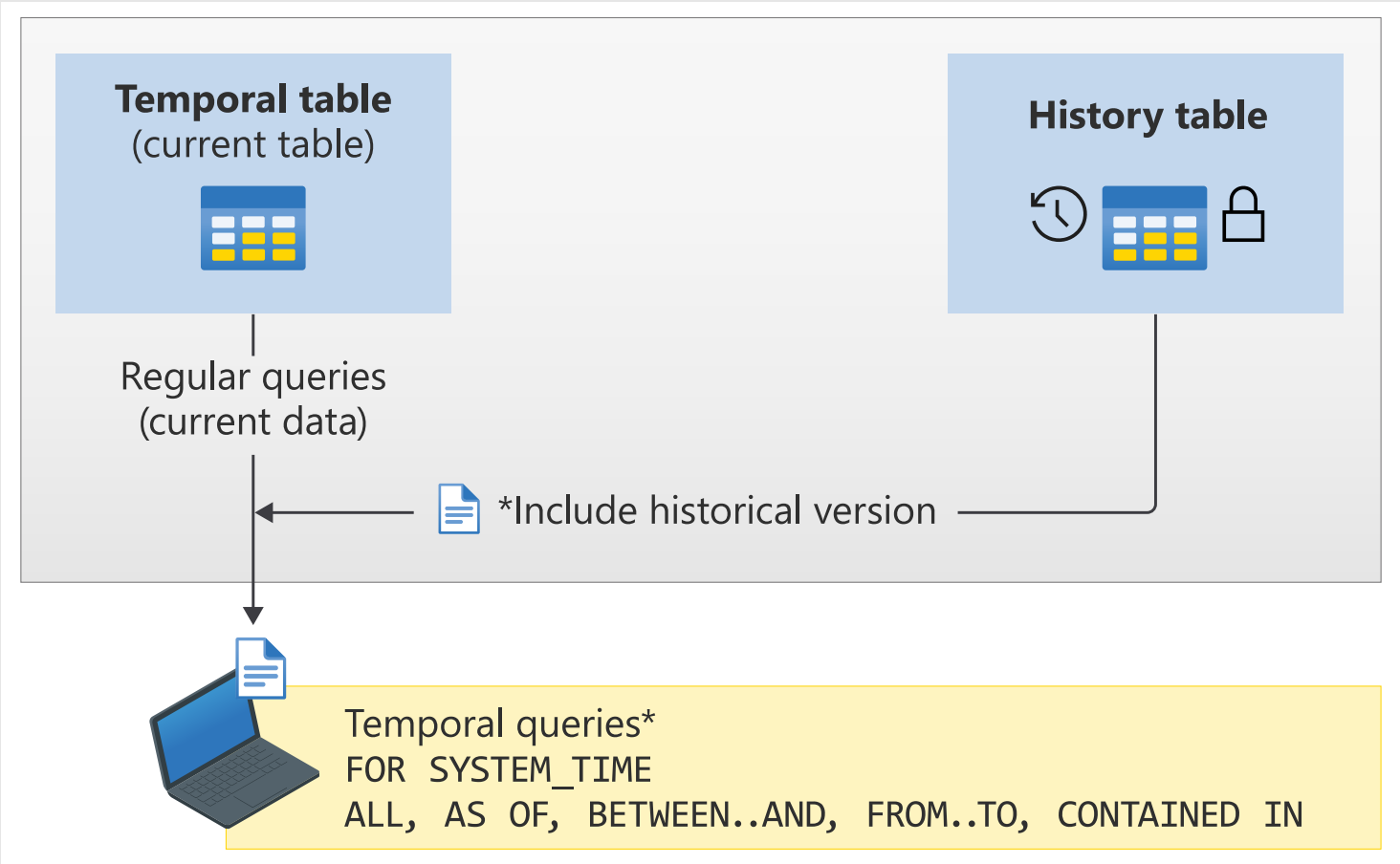
The times recorded in the system **datetime2** columns are based on the begin time of the transaction itself. For example, all rows inserted within a single transaction have the same UTC time recorded in the column corresponding to the start of the `SYSTEM_TIME` period.

When you run any data modification queries on a temporal table, the Database Engine adds a row to the history table, even if no column values change.

How do I query temporal data?

The `SELECT ... FROM <table>` statement has a new clause `FOR SYSTEM_TIME`, with five temporal-specific subclauses to query data across the current and history tables. This new `SELECT` statement syntax is supported directly on a single table, propagated through multiple joins, and through views on top of multiple temporal tables.

When you query using the `FOR SYSTEM_TIME` clause using one of the five subclauses, *historical* data from the temporal table are included, as shown in the following image.



The following query searches for row versions for an employee with the filter condition `WHERE EmployeeID = 1000` that were active at least for a portion of period between January 1, 2021 and January 1, 2022 (including the upper boundary):

SQL

```
SELECT * FROM Employee
    FOR SYSTEM_TIME
        BETWEEN '2021-01-01 00:00:00.0000000' AND '2022-01-01
00:00:00.0000000'
            WHERE EmployeeID = 1000 ORDER BY ValidFrom;
```

`FOR SYSTEM_TIME` filters out rows that have a period of validity with zero duration (`ValidFrom = ValidTo`).

Those rows are generated if you perform multiple updates on the same primary key within the same transaction. In that case, temporal querying returns only row versions before the transactions, and current rows after the transactions.

If you need to include those rows in the analysis, query the history table directly.

In the following table, `ValidFrom` in the Qualifying rows column represents the value in the `ValidFrom` column in the table being queried and `ValidTo` represents the value in the `ValidTo` column in the table being queried. For the full syntax and for examples, see FROM clause plus JOIN, APPLY, PIVOT, and Query data in a system-versioned temporal table.

Expand table

Expression	Qualifying rows	Note
`AS OF` *date_time*	`ValidFrom <=` *date_time* `AND ValidTo >` *date_time*	Returns a table with rows containing the values that were current at the specified point in time in the past. Internally, a union is performed between the temporal table and its history table. The results are filtered to return the values in the row that was valid at the point in time, specified by the *date_time* parameter. The value for a row is deemed valid if the *system_start_time_column_name* value is less than or equal to the *date_time* parameter value, and the *system_end_time_column_name* value is greater than the *date_time* parameter value.
`FROM` *start_date_time* `TO` *end_date_time*	`ValidFrom <` *end_date_time* `AND ValidTo >` *start_date_time*	Returns a table with the values for all row versions that were active within the specified time range, regardless of whether they started being active before the *start_date_time* parameter value for the `FROM` argument or ceased being active after the *end_date_time* parameter value for the `TO` argument. Internally, a union is performed between the temporal table and its history table. The results are filtered to return the values for all row versions that were active at any time during the time range specified. Rows that stopped being active exactly on the lower boundary defined by the `FROM` endpoint aren't included, and records that became active exactly on the upper boundary defined by the `TO` endpoint are also not included.
`BETWEEN`	`ValidFrom <=`	Same as previous in the `FOR SYSTEM_TIME FROM` *start_date_time* `TO`

start_date_time AND *end_date_time*	*end_date_time* AND ValidTo > *start_date_time*	*end_date_time* description, except the table of rows returned includes rows that became active on the upper boundary defined by the *end_date_time* endpoint.
CONTAINED IN (*start_date_time*, *end_date_time*)	ValidFrom >= *start_date_time* AND ValidTo <= *end_date_time*	Returns a table with the values for all row versions that were opened and closed within the specified time range defined by the two period values for the CONTAINED IN argument. Rows that became active exactly on the lower boundary or ceased being active exactly on the upper boundary are included.
ALL	All rows	Returns the union of rows that belong to the current and the history table.

Hide the period columns

You can choose to hide the period columns, such that queries that don't explicitly reference them don't return these columns (for example, when running `SELECT * FROM <table>`).

To return a hidden column, you must explicitly refer to the hidden column in the query. Similarly `INSERT` and `BULK INSERT` statements continue as if these new period columns weren't present (and the column values are auto-populated).

For details on using the `HIDDEN` clause, see CREATE TABLE and ALTER TABLE.

Samples

- **ASP.NET**: See the ASP.NET Core web application to learn how to build a temporal application using temporal tables.
- **AdventureWorks sample database**: Download the AdventureWorks database for SQL Server, which includes temporal table features.

Related content

- Temporal table considerations and limitations

- Manage retention of historical data in system-versioned temporal tables
- Partition with temporal tables
- Temporal table system consistency checks
- Temporal table security
- Temporal table metadata views and functions
- Work with memory-optimized system-versioned temporal tables
- Create a system-versioned temporal table
- Modify data in a system-versioned temporal table
- Query data in a system-versioned temporal table
- Get started with system-versioned temporal tables
- System-versioned temporal tables with memory-optimized tables
- Getting started with temporal tables in Azure SQL

Last updated on 07/20/2026